

OMI Doctrine: Notation as Cipher, Cipher as Notation

Lisp teaches:

```
code as data  
data as code
```

POSIX teaches:

```
everything is a file
```

OMI teaches:

```
notation as cipher  
cipher as notation
```

The Difference Between Code and Cipher

Conventionally:

```
code
```

operates on meaning.

A code replaces:

```
word  
phrase  
sentence
```

with another symbolic representation.

Meanwhile:

cipher

operates on structure.

A cipher rewrites:

bits
bytes
characters
patterns

without requiring semantic understanding.

This distinction is fundamental.

Code acts on meaning.

Cipher acts on form.

OMI Removes The Boundary

OMI removes the distinction.

A notation is a cipher.

A cipher is a notation.

Changing notation changes interpretation.

Changing interpretation changes notation.

The underlying binary source remains unchanged.

Only the rewrite surface changes.

The Binary Source Never Changes

Traditional systems assume:

data changes

OMI assumes:

interpretation changes

A binary source of truth may remain identical.

The same bit pattern may be interpreted as:

text
address
instruction
receipt
graph
proof
nomogram
rewrite table

depending on the notation applied.

Thus:

notation = cipher

because notation rewrites interpretation.

Omicron As Cipher Selector

The Omicron is not a value.

The Omicron is a selector.

It chooses which notation is active.

This is analogous to how a cipher selects a decoding surface.

omi-

and

-imo

represent dual readings of the same structure.

The bits remain.

The interpretation rotates.

The Rewrite Surface

OMI does not begin with objects.

OMI begins with rewrite surfaces.

A rewrite surface is any structure capable of changing interpretation:

endianness
BCD
UTF
UTF-EBCDIC
escape sequences
BOM markers
control characters
frame headers
BiDi markers

Each of these acts like a cipher.

None necessarily modify the underlying data.

They modify how the data is read.

Control Characters As Cipher Operators

The ASCII control range:

0x00..0x1F

is traditionally described as non-printing characters.

OMI treats them as notation operators.

They define:

```
start
stop
shift
escape
acknowledge
separate
return
advance
```

These are not symbols.

They are interpretation transitions.

They are cipher operations.

Unicode As Notation Space

Unicode is usually described as a character space.

OMI treats Unicode as notation space.

Each code point becomes:

a possible interpretation boundary

rather than merely:

a printable glyph

Private-use regions become especially useful because they allow notation without conflicting with public meaning.

Cipher Without Secrecy

Cryptography usually uses ciphers for secrecy.

OMI uses ciphers for interpretation.

The goal is not:

hide meaning

The goal is:

declare meaning

The same mechanism is used.

The purpose differs.

BiDi As Dual Reading

BiDi is not merely left-to-right versus right-to-left.

BiDi expresses the deeper principle that:

one structure
may have multiple lawful readings

The same source can be interpreted:

forward
backward
mirrored
folded
projected

without changing the source itself.

The OMI Principle

Traditional computing says:

data is transformed by code

OMI says:

notation transforms interpretation

and therefore:

notation is cipher
cipher is notation

The source remains.

The reading changes.

The rewrite is the computation.